

39. 组合总和

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium	(Not Available)	(Not Available)	-	-	0

- **Tags:** 数组, 回溯
- **Companies:** (Not Available)

给你一个 无重复元素的整数数组 `candidates` 和一个目标整数 `target`，找出 `candidates` 中可以使数字和为目标数 `target` 的所有 不同组合。

`candidates` 中的 同一个数字可以 无限制重复被选取。如果至少一个数字的被选数量不同，则两种组合是不同的。

对于给定的输入，保证和为 `target` 的不同组合数少于 150 个。

示例 1:

输入: `candidates = [2,3,6,7]`, `target = 7`

输出: `[[2,2,3],[7]]`

解释:

2 和 3 可以形成一组候选， $2 + 2 + 3 = 7$ 。注意 2 可以使用多次。

7 也是一个候选， $7 = 7$ 。

仅有这两种组合。

示例 2:

输入: `candidates = [2,3,5]`, `target = 8`

输出: `[[2,2,2,2],[2,3,3],[3,5]]`

示例 3:

输入: `candidates = [2]`, `target = 1`

输出: `[]`

提示:

- `1 <= candidates.length <= 30`
- `2 <= candidates[i] <= 40`
- `candidates` 的所有元素 互不相同
- `1 <= target <= 40`

Discussion | Solution

解决方法

1. 问题理解

目标：从给定数组 `candidates` (无重复元素) 中找出所有不同的组合，使得组合中元素的和等于目标值 `target`。关键点：- `candidates` 中的元素可以无限次重复使用。- 组合是不同的，如果至少一个数字的选择数量不同 (例如 `[2,2,3]` 和 `[2,3,3]` 是不同的)。顺序不影响组合的唯一性 (例如 `[2,2,3]` 和 `[2,3,2]` 视为相同组合)。输入：- `candidates`: 整数数组。- `target`: 目标整数。输出：包含所有满足条件的组合的列表。

2. 思路分析

这仍然是一个回溯问题，与“子集”问题类似，但有目标和的限制，并且元素可以重复使用。

决策树模型：- 树的每个节点代表一个构建中的组合。- 从一个节点出发，可以尝试将 `candidates` 中的某个数（或其本身）加入组合。

回溯过程：

1. 路径 (**Path**): 记录当前已经选择的数字构成的组合 `current_combination`。
2. 选择列表 (**Choices**): 在当前步骤，可以从 `candidates` 中选择哪些数字。为了避免产生重复的组合（如 `[2,3]` 和 `[3,2]` 算作同一个组合，因为顺序无关），我们需要确保选择的有序性。
3. 目标与约束：当前组合的和 `current_sum` 不能超过 `target`。
4. 结束条件：当 `current_sum` 等于 `target` 时，找到了一个有效的组合。

具体步骤：

1. (可选但推荐) 排序：对 `candidates` 数组进行排序。这有助于后续的剪枝操作。
2. 定义回溯函数 `backtrack(start_index, current_sum, current_combination)`:
 - `start_index`: 本轮选择应该从 `candidates` 数组的哪个索引开始。这非常重要，用于避免重复组合。例如，如果我们已经考虑过 `candidates[i]`，那么在后续的递归中，我们只应考虑 `candidates[i]` 及之后的元素，防止生成 `[2,3]` 和 `[3,2]` 这样的重复组合。
 - `current_sum`: 当前组合 `current_combination` 中所有元素的和。
 - `current_combination`: 当前构建的部分组合 (列表)。
3. 递归终止条件/剪枝：
 - 如果 `current_sum == target`:
 - 找到了一个有效的组合。将 `current_combination` 的副本加入结果列表 `result`。
 - 返回 (因为不能再添加任何正数了)。
 - 如果 `current_sum > target`:
 - 当前路径无效，无法达到目标和。
 - 返回 (剪枝)。
4. 遍历选择列表：

- 从 `start_index` 开始，遍历 `candidates` 数组。设当前索引为 `i`。
 - 获取当前候选数字 `num = candidates[i]`。
 - (剪枝优化，需要先排序)：如果 `current_sum + num > target`，由于数组已排序，后续的 `candidates[j]` ($j > i$) 只会更大，`current_sum + candidates[j]` 也必然大于 `target`。因此，可以直接 `break` 循环，停止当前层级的搜索。
 - 做选择：将 `num` 加入 `current_combination`。
 - 递归进入下一层决策：调用 `backtrack(i, current_sum + num, current_combination)`。
 - 关键点：递归调用的 `start_index` 仍然是 `i`，而不是 `i + 1`。这表示当前数字 `candidates[i]` 可以在下一轮递归中被重复选择。
 - 撤销选择 (回溯)：将 `num` 从 `current_combination` 末尾移除 (`current_combination.pop()`)，恢复状态，以便尝试包含 `candidates[i+1]` 等其他分支。
5. 主函数调用：
- 初始化结果列表 `result = []`。
 - 对 `candidates` 排序 (如果需要剪枝优化)。
 - 调用 `backtrack(0, 0, [])` 开始回溯。
 - 返回 `result`。

3. 代码实现 (Python)

```
from typing import List

class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]:
        """
        使用回溯算法找出所有和为 target 的组合 (元素可重复使用)。
        Args:
            candidates: 无重复元素的整数数组。
            target: 目标整数。
        Returns:
            包含所有满足条件的组合的列表。
        """
        result = [] # 存储最终结果
        path = []   # 存储当前构建的组合路径

        # 对 candidates 排序，便于后续剪枝
        candidates.sort()
        n = len(candidates)

        def backtrack(start_index: int, current_sum: int):
            """
            回溯函数核心逻辑。
            Args:
                start_index: 本轮选择的起始索引。
            """
```

```

        current_sum: 当前路径的和。
    """
    # 递归终止条件 1: 和达到目标值
    if current_sum == target:
        result.append(path[:]) # 添加路径副本
        return

    # 递归终止条件 2 (隐式包含在循环判断中): 和超过目标值
    # 或者在循环内部进行剪枝判断

    # 遍历选择列表 (从 start_index 开始)
    for i in range(start_index, n):
        num = candidates[i]

        # 剪枝: 如果当前和加上候选数已经超过 target,
        # 由于数组已排序, 后续の数更大, 无需继续尝试
        if current_sum + num > target:
            break

        # --- 做选择 ---
        path.append(num)

        # --- 递归进入下一层决策 ---
        # 注意: 下一个起始索引仍然是 i, 因为元素可以重复使用
        backtrack(i, current_sum + num)

        # --- 撤销选择 (回溯) ---
        path.pop()

    # 从索引 0, 和 0, 空路径开始回溯
    backtrack(0, 0)
    return result

# 示例调用
sol = Solution()
print(sol.combinationSum(candidates = [2,3,6,7], target = 7))
print(sol.combinationSum(candidates = [2,3,5], target = 8))
print(sol.combinationSum(candidates = [2], target = 1))

```

4. 复杂度分析

设 *candidates* 数组的长度为 *N*, 目标值为 *T*。

- 时间复杂度: $O(N * 2^T)$ 或更宽松的上界 $O(N * S)$, 其中 *S* 是解的数量。
 - 这类问题的精确时间复杂度分析比较困难。状态树的节点数可能非常多。
 - 如果不考虑剪枝, 每个位置可以选 *N* 个数, 深度可能达到 *T* /

- min(candidates)。
- 排序需要 $O(N \log N)$ 。
- 每次找到解时复制路径需要时间，最长路径长度约为 $T / \min(\text{candidates})$ 。
- 一个不严格的上界可以是 $O(N * \text{number_of_solutions})$ ，因为每个解都需要构建出来。题目保证了解的数量少于 150，但这只是特定输入的约束。更通用的分析倾向于指数级，与 T 和 N 相关。
- 考虑剪枝后，实际性能会好很多，但最坏情况分析仍然复杂。可以粗略认为是指数级别的 $O(N * K)$ ， K 是搜索树的节点数。
- 空间复杂度： $O(T)$ 或 $O(\text{target} / \min_candidate)$
 - 主要是递归调用栈的深度。在最坏情况下，如果 `candidates` 中有 1，递归深度可以达到 T 。
 - `path` 列表也需要类似的空间。
 - 结果列表 `result` 的空间取决于解的数量和解的平均长度，不计入算法本身空间复杂度。

5. 如何在面试中讲解

1. 问题理解：
 - “目标是从 `candidates` 数组中找到所有和为 `target` 的组合。关键在于，数组中的元素可以无限次重复使用，且组合的顺序不重要。”
2. 核心思路：回溯：
 - “这是一个组合问题，适合用回溯法解决。我们尝试构建和为 `target` 的组合。”
 - “定义回溯函数 `backtrack(startIndex, currentSum, currentPath)`。”
3. 回溯过程：
 - “状态变量： `startIndex` 用于控制选择范围，避免重复组合；`currentSum` 记录当前和；`currentPath` 记录当前组合。”
 - “结束条件/剪枝：”
 - “如果 `currentSum == target`，找到一个解，复制 `currentPath` 加入结果集，返回。”
 - “如果 `currentSum > target`，此路不通，返回。”
 - “选择与递归：遍历 `candidates` 从 `startIndex` 开始的元素 `num`。”
 - “(推荐) 剪枝：如果先对 `candidates` 排序，当 `currentSum + num > target` 时，可以直接 `break` 循环，因为后续元素更大。”
 - “选择 `num`：将其加入 `currentPath`。”
 - “递归调用 `backtrack(i, currentSum + num, currentPath)`。注意这里是 `i` 而不是 `i+1`，因为允许重复使用当前元素。”
 - “撤销选择：将 `num` 从 `currentPath` 中移除。”
4. 避免重复组合的关键：
 - “参数 `startIndex` 非常重要。它确保在递归的每一层，我们只考虑当前索引及之后的元素。例如，如果我们选择了 `candidates[i]`，下一层递归仍然可以从 `candidates[i]` 开始选，但再往后的选择不会回头选 `candidates[i-1]` 等，这样就避免了因顺序不同而产生的重复组合（如 `[2,3]` 和 `[3,2]`）。”
5. 初始调用与排序：

- “建议先对 `candidates` 排序以启用剪枝优化。”
 - “初始调用 `backtrack(0, 0, [])`。”
6. 复杂度分析:
- “时间复杂度是指数级的, 与 `candidates` 数量 `N` 和 `target` `T` 相关, 大致可以认为是 $O(N * K)$, `K` 是搜索树节点数。排序需要 $O(N \log N)$ 。”
 - “空间复杂度主要由递归深度决定, 最坏情况下可能与 `target` 相关, 为 $O(T)$ 。”